ELEC 442
Lab 1

LEARNING GOALS

Forward Kinematics

- *Spatial Mapping Principles*: Discover how geometric constraints reduce 3D spatial transformations down to the four unique parameters of the Denavit–Hartenberg convention.
- *Homogeneous Representations*: Grasp how 4×4 homogenous transformation matrices unify rotation and translation to seamlessly map configuration space to task space.
- *Hardware Boundaries*: Appreciate the physical and software limits of robotic manipulators, including joint limits and workspace restriction volumes.

Velocity Kinematics

- *Differential Kinematics*: Observe the role of the Jacobian and its inverse in mapping rates of change between joint variables and end-effector motion.
- *Limitations of Numerical Integration*: Recognize why digital systems suffer from truncation and round-off error, and how cumulative drift compromises open-loop control.
- *Continuity in Motion Planning*: Evaluate how boundary conditions shape trajectory profiles, and understand why continuity in lower-order derivatives (position, velocity) can still result in harsh higher-order discontinuities (jerk).

1 Introduction

1.1 The QArm

Our instrument of choice for this term is the Quanser QArm, a four-degree-of-freedom manipulator that you will work with in teams of two or three. The four joints are the base, the shoulder, the elbow, and the wrist. You can also open and close the gripper. Lastly, a RealSense D415 RGB and depth camera is included, which will be used later in the term for computer vision applications.

i Note: Joint angles as an array/list will always be given or expected to be recieved in the order: base, shoulder, elbow, wrist.



Figure 1: Quanser QArm - https://www.quanser.com/products/qarm/

Joint Specifications									
#	**Item**		**Base**	**Shoul-der**	**Elbow**	**Wrist**	**Units**	**Grip-per**	**Units**
1	Joint range	Min.	-170	-85	-95	-160	degrees	0	%
		Max	$+170$	$+85$	$+75$	$+160$		100	
2	Max. speed		$\pm\frac{\pi}{2}$	$\pm\frac{\pi}{2}$	$\pm\frac{\pi}{2}$	$\pm\frac{\pi}{2}$	rad/s	±100	%/s
3	Max. acceleration		$\pm\frac{\pi}{3}$	$\pm\frac{\pi}{3}$	$\pm\frac{\pi}{3}$	$\pm\frac{\pi}{3}$	rad/s/s	±400	%/s/s
4	Current	Instantaneous	4.4	8.8	4.4	2.3	Amps	1.4	Amps
		Continuous	1.1	2.2	1.1	0.6		0.35	
5	Torque	Instantaneous	10.6	21.2	10.6	4.1	Nm	1.9	Nm
		Continuous	2.65	5.3	2.65	1.0		0.5	

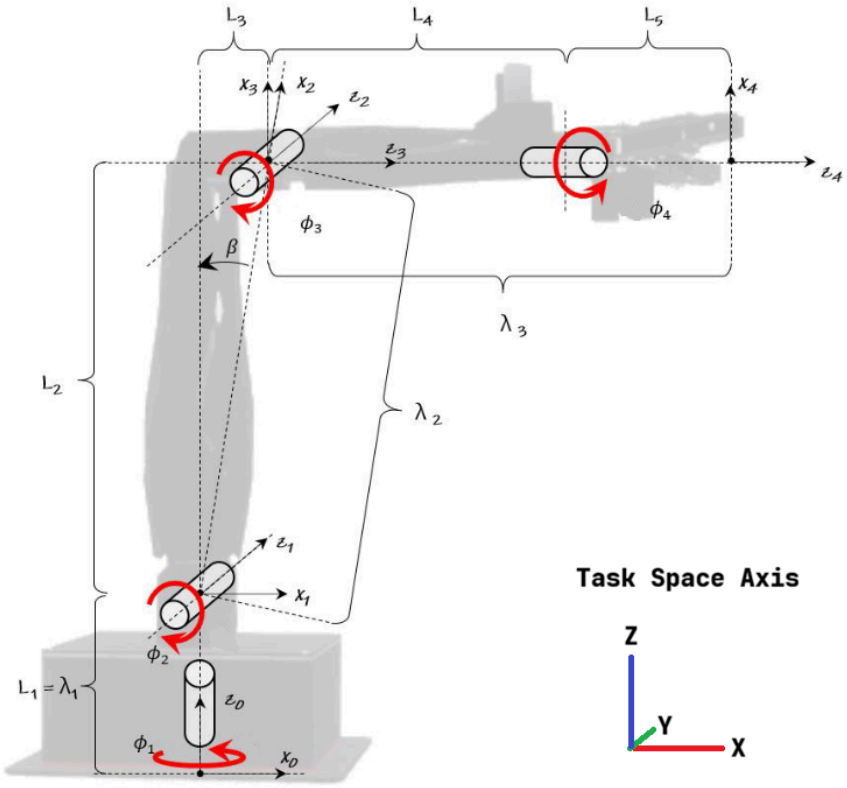
Table 1: Quanser QArm Specifications

2 Concept Review

2.1 Denavit–Hartenberg Convention

In a general three-dimensional space, describing the relative position and orientation between two coordinate frames requires six independent parameters: three for translation and three for rotation. However, by imposing specific geometric constraints on how we attach coordinate frames to robot links, the Denavit–Hartenberg (DH) convention reduces this mapping to just four parameters per joint. The transformation from frame $i-1$ to frame i is uniquely defined by executing the following four operations in sequence:

- θ_i: Rotation about $\underline{k}_{i-1}$ to align the $\underline{i}_{i-1}$ direction with the $\underline{i}_i$ direction.
- d_i: Translation along $\underline{k}_{i-1}$ to align the $\underline{i}_{i-1}$ and $\underline{i}_i$ axes.
- a_i: Translation along $\underline{i}_i$ to complete the shift to the new origin.
- α_i: Rotation about $\underline{i}_i$ to align the $\underline{k}_{i-1}$ axis with the $\underline{k}_i$ axis.



Parameter	Value	Parameter	Value
L_1	0.14 m	L_4	0.25 m
L_2	0.35 m	L_5	0.15 m
L_3	0.05 m	β	8.13°

Figure 2: QArm dimensions and values

2.2 Forward Kinematics Introduction

Forward kinematics allows us to determine the end-effector's pose (position and orientation) in task space from a given set of joint coordinates in configuration space. To compute this efficiently, we use 4×4 homogeneous transformation matrices. These matrices consolidate a 3×3 rotation matrix and a 3×1

translation vector into a single operator, applying both rotation and translation simultaneously through a single matrix multiplication. A general homogeneous transformation matrix, T, is defined as:

$$T = \begin{pmatrix} \boldsymbol{R} & \underline{\boldsymbol{o}} \\ \underline{\mathbf{0}}_{1\times 3} & 1 \end{pmatrix} = \begin{pmatrix} r_{11} & r_{12} & r_{13} & x \\ r_{21} & r_{22} & r_{23} & y \\ r_{31} & r_{32} & r_{33} & z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Where:

- $\boldsymbol{R}$ in $\mathbb{R}^{\{3\times 3\}}$ is the orthogonal rotation matrix representing the orientation of the new frame relative to the reference frame.
- $\underline{\boldsymbol{o}}$ in $\mathbb{R}^{\{3\times 1\}}$ is the translation vector representing the position coordinates (x, y, z) of the new frame's origin expressed in the reference frame.
- $\underline{\mathbf{0}}_{\{1\times 3\}}$ is a row vector of zeros mapping the perspective transformation.

For your convenience, the following is provided, however review your class notes for more information:

$$\begin{pmatrix} \underline{C}_i & \underline{o}_i \\ 0^T & 1 \end{pmatrix} = \begin{pmatrix} \underline{C}_{i-1} & \underline{o}_{i-1} \\ 0^T & 1 \end{pmatrix} \begin{pmatrix} e^{\theta_i k\times} & 0 \\ 0^T & 1 \end{pmatrix} \begin{pmatrix} I & d_i k \\ 0^T & 1 \end{pmatrix} \begin{pmatrix} I & a_i k \\ 0^T & 1 \end{pmatrix} \begin{pmatrix} e^{\alpha_i i\times} & 0 \\ 0^T & 1 \end{pmatrix} = \begin{pmatrix} \underline{C}_{i-1} & \underline{o}_{i-1} \\ 0^T & 1 \end{pmatrix} {}^{i-1}T_i$$

Another way to represent ${}^{i-1}T_i$:

$${}^{i-1}T_i = \begin{pmatrix} \cos\theta_i & -\sin\theta_i \cos\alpha_i & \sin\theta_i \sin\alpha_i & a_i \cos\theta_i \\ \sin\theta_i & \cos\theta_i \cos\alpha_i & -\cos\theta_i \sin\alpha_i & a_i \sin\theta_i \\ 0 & \sin\alpha_i & \cos\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

2.3 Velocity Kinematics Introduction

One of the most powerful operators in robotics is the Jacobian matrix. While the primary focus of this lab is utilizing the Jacobian for trajectory generation, it is also fundamentally used to map task-space forces to joint torques, resolve kinematic redundancy, and identify kinematic singularities.

2.3.1 Generating Joint Trajectories

The Jacobian relates joint velocities to end-effector linear and angular velocities through the following relationship:

$$\underline{\dot{q}} = \boldsymbol{J}^{-1}\big(\underline{q}\big)\underline{v}$$

Where $\underline{\dot{q}}$ represents the joint velocities, $\boldsymbol{J}^{-1}\big(\underline{q}\big)$ is the inverse Jacobian matrix dependent on the current joint configurations $\underline{q}$, and $\underline{v}$ is the desired task-space velocity vector.

Using the robot's current position as an initial condition, we can use the inverse Jacobian and a desired task-space velocity vector to numerically integrate and generate a continuous joint trajectory.

For example, this can be implemented in Python as follows:

```
dt = current_time - previous_time
d_joint = J_inv * d_trajectory
joint_pos += d_joint * dt
```

2.3.2 Creating a Trajectory

When designing a trajectory for the manipulator to follow, several critical factors must be considered:

- Must the end-effector pass through specific via-points or targets?
- Are there physical obstacles in the workspace to avoid?

- What are the operational limits on the joint motor torques?
- What are the geometric constraints of the robot's reachable workspace?

While numerous methods exist to smoothly transition from one task-space position to another, a common and straightforward approach is to utilize a cubic spline. A cubic polynomial profile provides four degrees of freedom (coefficients). By specifying four boundary conditions—typically the initial and final positions, along with setting the initial and final velocities to zero ($\underline{v} = 0$)—we can generate a trajectory with continuous position and velocity profiles.

By evaluating this cubic spline position function at each time step, we can also actively compensate for numerical integration drift. This is achieved by comparing the robot's calculated position against the desired spline target and introducing a feedback correction factor.

For example, this tracking and correction loop can be implemented in Python as follows:

```
v_cmd = spline_velocity(t) + Kp*(spline(t)-location)
```

Where:
- `v_cmd` is the dessired task space velocity
- `spline_velocity` is the time derivative of `spline` (found by applying `.derivative()`), takes time as an input
- `spline` is the cubic spline object, takes time as an input
- `Kp` is a constant to be tuned
- `location` is the task space location of the EE (found using forward kinematics)

2.3.3 Analytic Jacobian

You are likely familiar with the **Geometric Jacobian** ($\boldsymbol{J}_G \in \mathbb{R}^{6\times n}\}$) from class lectures. The geometric Jacobian maps the joint velocities directly to the 3 linear velocity components and the 3 angular velocity components of the end-effector framework:

$$\boldsymbol{J}_i = \begin{cases} \begin{pmatrix} \underline{k}_{i-1} \times (\underline{o}_n - \underline{o}_{i-1}) \\ \underline{k}_{i-1} \end{pmatrix} & \text{if joint } \ i \ \text{ is revolute} \\ \begin{pmatrix} \underline{k}_{i-1} \\ \underline{0} \end{pmatrix} & \text{if joint } \ i \ \text{ is prismatic} \end{cases}$$

The **Analytic Jacobian** ($\boldsymbol{J}_A$), on the other hand, relates the joint velocities directly to a designated set of chosen *task space variables*, $\underline{x}$. For our upcoming lab objectives, we will specify our task space using 4 coordinates: the 3 linear coordinate positions (p_x, p_y, p_z) along with a single wrist orientation angle (φ). In this lab, we will be using the analytic Jacobian for velocity kinematics as it's generally much more convenient to work with since we mostly care about linear velocities.

By employing the multivariate chain rule to evaluate the instantaneous rate of change for these specific variables over time:

$$\frac{d\underline{x}}{dt} = \frac{\partial \underline{x}}{\partial \underline{\theta}} \frac{\partial \underline{\theta}}{dt} \rightarrow \dot{\underline{x}} = \boldsymbol{J}_A(\underline{\theta})\dot{\underline{\theta}}$$

This formulation yields the following explicit matrix structure for our analytic Jacobian:

$$\boldsymbol{J}_A(\underline{\theta}) = \begin{pmatrix} \frac{\partial p_x}{\partial \theta_1} & \frac{\partial p_x}{\partial \theta_2} & \frac{\partial p_x}{\partial \theta_3} & \frac{\partial p_x}{\partial \theta_4} \\ \frac{\partial p_y}{\partial \theta_1} & \frac{\partial p_y}{\partial \theta_2} & \frac{\partial p_y}{\partial \theta_3} & \frac{\partial p_y}{\partial \theta_4} \\ \frac{\partial p_z}{\partial \theta_1} & \frac{\partial p_z}{\partial \theta_2} & \frac{\partial p_z}{\partial \theta_3} & \frac{\partial p_z}{\partial \theta_4} \\ \frac{\partial \varphi}{\partial \theta_1} & \frac{\partial \varphi}{\partial \theta_2} & \frac{\partial \varphi}{\partial \theta_3} & \frac{\partial \varphi}{\partial \theta_4} \end{pmatrix}$$

Implementation Highlights & Trade-offs

- By purposefully reducing our operational task space to exactly 4 variables matching our 4-DOF manipulator configuration, $\boldsymbol{J}_A$ becomes a clean 4×4 square matrix. This enables standard matrix inversion ($\boldsymbol{J}_A^{-1}$) for direct inverse kinematics trajectories, bypassing the need for pseudo-inverses.
- Ditching the full 3D rotation representation means we forfeit the ability to explicitly extract the absolute physical angular velocity vector ($\boldsymbol{\omega}$). However, because our lab focus centers tightly on targeted linear pathways and a single, predictable orientation, this compromise is well worth it.

---

3 Pre-lab

3.1 Pre-lab Setup

For personal software setup and usage instructions: Lab 0 Template Repo

For Lab 1 setup, instructions and files: Lab 1 Template Repo

3.1.1 Pre-lab Setup Checklist

Before walking into the lab, ensure you have:

- [ ] Installed Git, Python and VSCode
- [ ] Created a new repo using the Lab-1 repo as a template and cloned to your device
- [ ] Skimmed the QArm anatomy, joint limits, and DH parameter conventions in Figure 1, Table 1 and Figure 2.

Optional Extras:

- [ ] Installed QLabs and created your Quanser account.

3.2 Pre-lab Exercises

Complete these before entering the lab:

3.2.1 DH Parameters & Table

Exercise 1:

Fill out the following DH table using the QArm values:

Link	θ_i	d_i	a_i	α_i
1				
2				
3				
4				

3.2.2 Forward Kinematics Implementation

Using the joint positions and your derived homogeneous transformation matrix, implement a forward kinematics function in the following form.

```
location, rotation = forward_kin(phi)
```

Role	Name	Dimension	Description
input	`phi`	4	Joint angles in order from base to wrist
output	`location`	3	Location of the {4} origin in the {0} coordinate system
	`rotation`	3×3	Representing rotation from {4} (gripper tip) to {0} (base) frame

Tip: Create each {i} to {i+1} transformation matrix using your DH table (perhaps with a helper function), then multiply them together.

Tip: Both the location vector and rotation matrix can be extracted directly from the same homogeneous transformation matrix.

Exercise 2: Implement the forward kinematics function `forward_kin(phi)` using your DH table. Test your function against the expected outputs below, debug any issues that arise, and record your results for the test cases:

Input (degrees)	Location [x, y, z]	Rotation Matrix
`[0, 0, 0, 0]`	[0.45, 0.0, 0.49]	$\begin{pmatrix} 0.0 & 0.0 & 1.0 \\ 0.0 & -1.0 & 0.0 \\ 1.0 & 0.0 & 0.0 \end{pmatrix}$
`[30, 55, -60, 0]`	[0.62, 0.36, 0.33]	$\begin{pmatrix} -0.08 & 0.5 & 0.86 \\ -0.04 & -0.87 & 0.5 \\ 1.0 & 0.0 & 0.09 \end{pmatrix}$

Record your calculated outputs for the following configurations:

Input	Location [x, y, z]	Rotation Matrix
`[0, -11, -75, 0]`		
`[-90, 73.5, -75, -50]`		
`[-90, 73.5, -75, 0]`		
`[90, -85, 75, 90]`		

3.2.3 Velocity Kinematics Concept Check

Exercise 3:

Answer the following conceptual questions on velocity kinematics:

1. Why can't we directly command a robotic manipulator with Cartesian velocities in task space $(\dot{x}, \dot{y}, \dot{z})$ without an inverse Jacobian ($\underline{\dot{q}} = \boldsymbol{J}^{-1}\underline{v}$)? What physical variables do the robot's joint motors actually control?

2. For our 4-DOF QArm, why does restricting our task space to four variables $\left(p_x, p_y, p_z, \varphi\right)$ produce a square 4×4 analytic Jacobian matrix, and what computational benefit does this provide for real-time trajectory tracking compared to an arbitrary 6-DOF geometric Jacobian?

4 In-Lab Hardware Setup

4.1 Safety Warnings

The QArm's workspace limit will be enforced to protect the robot, surrounding equipment, and users. It will automatically stop moving if the limit is violated; however, just in case, please always keep an eye on the QArm during motion. In an emergency, keep your distance and, if safe, be prepared to turn off the power switch or terminate the program.

4.2 GitHub Repository Setup

1. Log in to GitHub and navigate to the template repository: Lab 1 GitHub Repository
2. Create a new repository using the provided template and add your group members as collaborators.
3. Clone your newly created repository to your computer (and onto the lab laptop) and open the folder in VS Code.

4.3 Hardware (Physical QArm) Setup

Please handle the QArm with care:

1. Ensure that the metal baseplate + QArm is on a stable, flat surface and that the QArm is mounted securely to the baseplate with the 4 corner thumb-screws.
2. Slot the cardboard base with the black side up under the front of the metal baseplate, lining up with the front feet of the baseplate.
3. Connect the QArm to the lab laptop by inserting the pair of USB cables ($1\times$ USB-A to USB-B, $1\times$ USB-A to USB-C) into the laptop and QArm
4. Plug in the 6-pin power connector securely and power on the QArm with the switch in the back.
   - You may need to turn the arm off at somepoint $\rightarrow$ this will cause the joints to go limp, make sure someone is ready to catch it
5. When launching any of the provided lab Python programs, you will be prompted to select your hardware target:
   - Enter `0` for the **Virtual QArm** (running in QLabs)
   - Enter `1` for the **Real/Physical QArm**

i Note: If you ran into any issues during the pre-lab virtual environment setup or QLabs account creation, let the instructor or TA know now so they can help resolve them while your team sets up the physical hardware.

5 Forward Kinematics Experiment

5.1 Pick and Place; Keyboard Controls

> *Congrats! You are just about to start a new position! Unfortunately, it was another terrible year for the job market, so instead of engineering, you decided to join the prestigious Museum of Impossibly Ridiculous and Fragile Objects (MIRFO) as an assistant night shift curator. MIRFO is filled with objects that make no sense, and worst (or best) of all: everything is just a breeze or a mistimed observation away from falling apart.*
>
> *Your very first task is to put the finishing touches on the brand-new "Ming Dynasty Porcelain Rubber Ducky" exhibit. Due to frustrating insurance requirements, humans are not allowed near the artifacts, so it is up to you and your trusty robotic arm to position the ducks into place for the grand opening tomorrow morning! Be very careful—the artifact is priceless, and one wrong move from your manipulator can quickly turn it into the "Ming Dynasty Pile of Shards."*

Open `QArm_keyboard_control.py` (Found in the `lab_machine_env` repo in `QArm-control`, or as a desktop shortcut on lab laptops). When executed, once focused on the pygame window, this script will allow you to manually control the position of the robot using your keyboard. Experiment with moving the arm around to get a feel for its workspace. The Pygame window will also display the live joint angles.

Use number keys [1] through [4] to select any of the 4 joints, then use the up and down arrow keys to move the joint in either direction. The home position will be approximately $(x, y, z) = (0.45, 0, 0.49)$ meters.

⚠️ **WARNING:** To prevent injuries and damage to the arm, it will be restricted to [restriction volume]; any attempt to leave this will cause the arm to lock in place. To reset, simply end and rerun the script; the arm should home then leave you in control again.

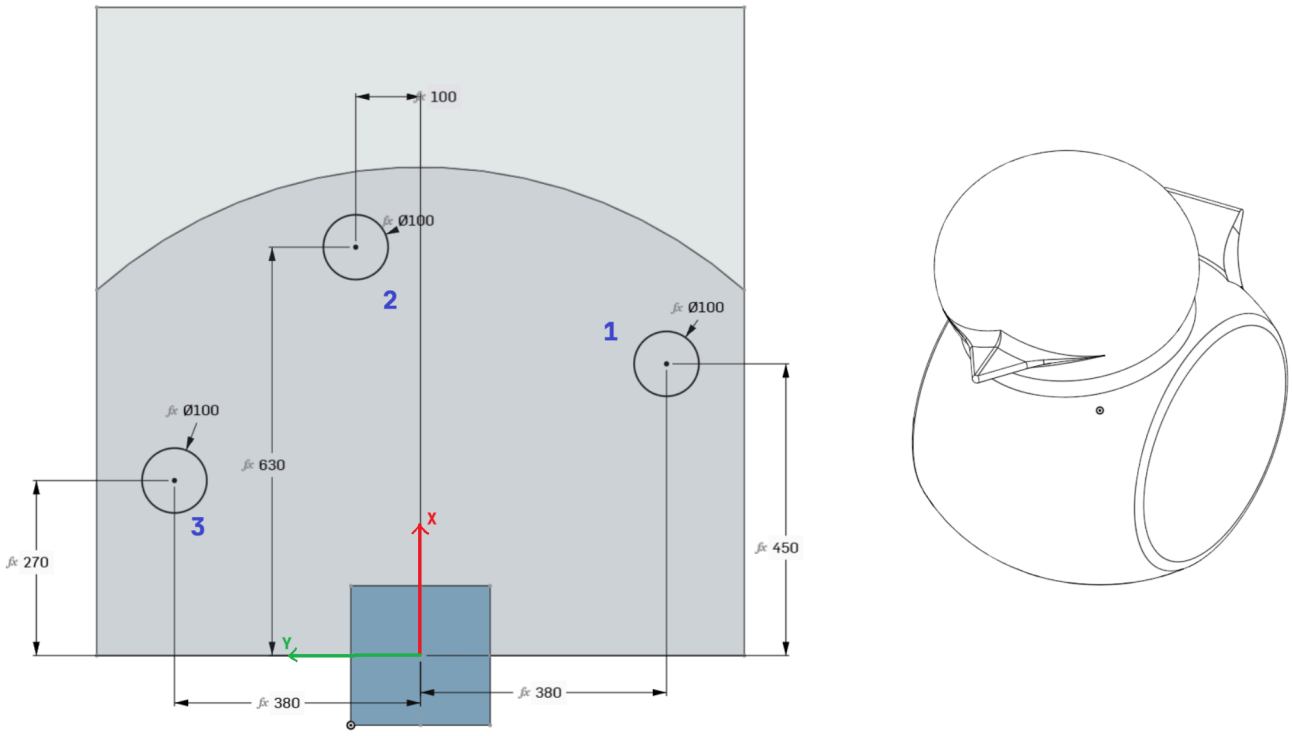


Figure 3: Zone diagram (left), Duck model (right)

Exercise 4:

Place the duck in zone 1. Manually move the duck into zone 2, release, then pick up and move to zone 3. Fill out joint positions at each point of interest as you go:

		Position			
		home	1	2	3
Joint	base				
	shoulder				
	elbow				
	wrist				

LEARNING GOALS

Forward Kinematics

✅ *Spatial Mapping Principles*: Discover how geometric constraints reduce 3D spatial transformations down to the four unique parameters of the Denavit–Hartenberg convention.

✅ *Homogeneous Representations*: Grasp how 4×4 homogenous transformation matrices unify rotation and translation to seamlessly map configuration space to task space.

✅ *Hardware Boundaries*: Appreciate the physical and software limits of robotic manipulators, including joint limits and workspace restriction volumes.

Velocity Kinematics

⬜ *Differential Kinematics*: Observe the role of the Jacobian and its inverse in mapping rates of change between joint variables and end-effector motion.

⬜ *Limitations of Numerical Integration*: Recognize why digital systems suffer from truncation and round-off error, and how cumulative drift compromises open-loop control.

⬜ *Continuity in Motion Planning*: Evaluate how boundary conditions shape trajectory profiles, and understand why continuity in lower-order derivatives (position, velocity) can still result in harsh higher-order discontinuities (jerk).

6 Velocity Kinematics Experiment

6.1 Demos

6.1.1 Visualize the purpose

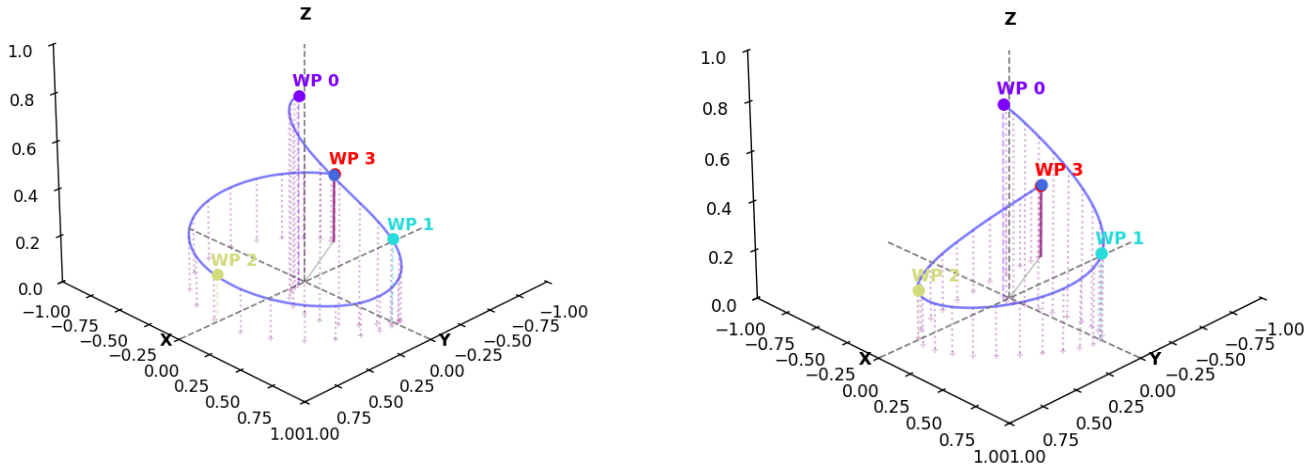


Figure 4: Joint space (left) vs task space (right) interpolation

Open and run `QArm_traj_demo.py`. This script moves the manipulator between the coordinates specified in `trajectory_points` using cubic spline interpolation in either joint space or task space. Figure 4 illustrates what the final output may look like; however, it is highly recommended to run the script yourself and observe the motion over time and from various angles. Also feel free to adjust the points by editing the `trajectory_points` list (note: trajectories that violate joint or workspace limits will terminate the script).

6.1.2 Exploring Integration Error

In an ideal scenario, integrating the Jacobian relationship would allow us to generate task-space trajectories continuously without ever needing to compute inverse kinematics—a significant computational advantage. Unfortunately, due to the discrete nature of digital computers and finite precision limits, each numerical iteration introduces truncation and round-off errors. Over time, this integration error accumulates to the point where the actual joint positions drift significantly from their theoretical targets, requiring external correction or recalibration.

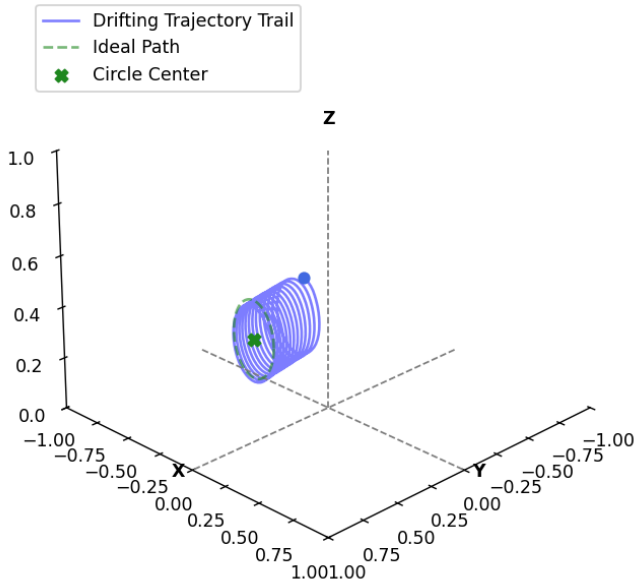


Figure 5: Accumulated integration error during a cyclic trajectory.

⚠️ **WARNING:** Although the script is designed to terminate automatically if the end-effector drifts more than 0.4 meters away from the center of the circle, numerical drift can cause erratic robotic movements. Keep the manipulator's workspace entirely clear and be prepared to terminate the script immediately if required.

Run the `integration_err.py` script to observe this phenomenon. As shown in Figure 5, the arm will attempt to continuously trace a circular path in the air. Observe how the trajectory drifts over successive cycles.

6.2 Trajectory Generation

> *Welcome back to the Museum of Impossibly Ridiculous and Fragile Objects (MIRFO)! Following your triumph with the Ming Dynasty Porcelain Rubber Ducks, management has promoted you to night-shift curator!*
>
> *Tonight's assignment is in the high-hazard wing: "The Chemical Vapor Barrier." A paper-thin containment wall of crystalline quartz separates a chamber of glowing, hyper-reactive chlorine trifluoride from the rest of the facility. To verify the barrier's structural integrity before tomorrow's grand opening, your QArm must perform an optical surface scan across the top ridge of the wall using its onboard camera (don't worry about writing computer vision code or turning the camera on tonight—we'll leave the image processing to the imagination!).*
>
> *Using task-space trajectory planning, smoothly sweep the arm's end-effector along the barrier. Be extremely careful: if your trajectory calculations dip and scrape the partition, the containment wall will shatter, vaporizing the gallery. No pressure!*

Note: To clear confusion, this is a separate exercise from the trajectory generation demo (`QArm_traj_demo.py`), though the same principles are used.

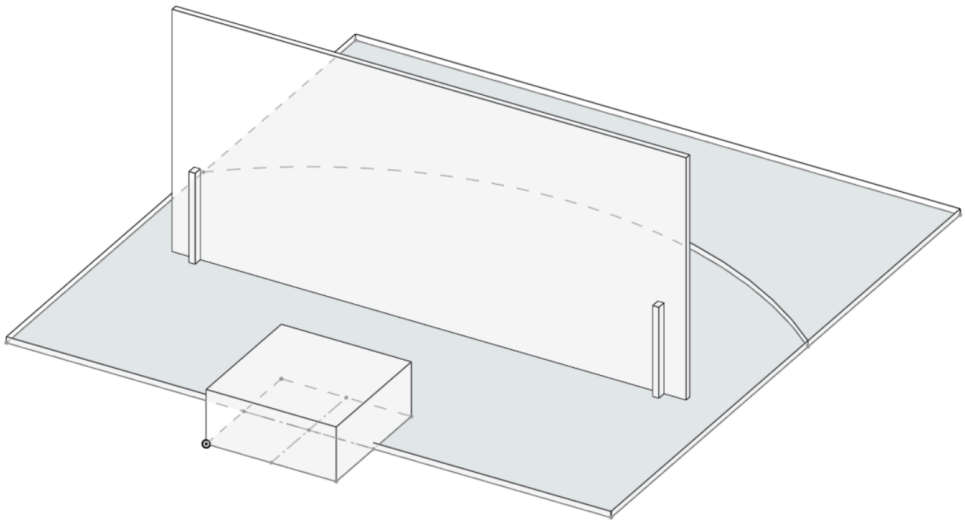


Figure 6: Environment Setup

A key component you will interact with is the `QArm_Lab_Interface` class. Key functions detailed below:

Method	Description	Inputs	Outputs
`write_to_arm(phi)`	Writes joint commands to the arm (in radians).	`phi` (in radians)	-
`read_from_arm()`	Reads the current joint positions from the arm (in radians).	-	`phi` (in radians)
`forward_kinematics(phi)`	Computes forward kinematics.	`phi` (in radians)	`p4` (vector from base to EE in {0} frame), `R4` (rotation matrix from {4} to {0} frame)
`Inv_Jacobian(phi)`	Computes the inverse of the analytic 4×4 Jacobian.	`phi` (in radians)	`J_inv` (Inverse of the Jacobian matrix)

Exercise 5:

Place the cardboard wall along the dotted gold line. The final result should have the arm smoothly sweep an (almost) ellipse ~5cm off of the barrier wall continuously. When completed, record the result (for submission) and show your working demonstration to the TA.

A trajectory with defined knot points is provided in `trajectory_generator.py`. Implement trajectory generation using the provided Jacobian function and `scipy.interpolate.CubicSpline`. The knot points are formatted as a 2D NumPy array (`trajectory_points`), where each sub-array contains three values representing the (x, y, z) coordinates expressed in the base {0} frame. Corresponding time stamps are in another numpy array (`t`), which is used to generate the spline.

The CubicSpline object has been initilized as `spline`, as well as it's derivative as `spline_vel`. Use time as an input to retrieve desired positions and velocities. Refer to the "Creating a Trajectory" section for more details.

Once complete, and if time permitting, feel free to adjust the knot points (and corresponding timestamps) to see different behaviours.

Reminders and Notes:

- Refer to "Velocity Kinematics Introduction" for more details and concept review
- Before multiplying `J_inv` with `v_cmd`, make sure to append a zero to `v_cmd` (representing a fixed wrist angle)

LEARNING GOALS

Forward Kinematics

✅ *Spatial Mapping Principles*: Discover how geometric constraints reduce 3D spatial transformations down to the four unique parameters of the Denavit–Hartenberg convention.

✅ *Homogeneous Representations*: Grasp how 4×4 homogenous transformation matrices unify rotation and translation to seamlessly map configuration space to task space.

✅ *Hardware Boundaries*: Appreciate the physical and software limits of robotic manipulators, including joint limits and workspace restriction volumes.

Velocity Kinematics

✅ *Differential Kinematics*: Observe the role of the Jacobian and its inverse in mapping rates of change between joint variables and end-effector motion.

✅ *Limitations of Numerical Integration*: Recognize why digital systems suffer from truncation and round-off error, and how cumulative drift compromises open-loop control.

✅ *Continuity in Motion Planning*: Evaluate how boundary conditions shape trajectory profiles, and understand why continuity in lower-order derivatives (position, velocity) can still result in harsh higher-order discontinuities (jerk).



---

7 Post-lab

Exercise 6:

Explore what happens if we "artificially" introduce some error in accuracy or precision. This can manifest in several ways:

- Joint angle error due to incorrect measurement or backlash/slip
- Link measurement error due to manufacturing issues or material strain
- Signal or sensor noise

Calculate and record the task space positional error (magnitude) in several poses when we purposefully introduce error using your forward kinematics implementation.

`Pose_1 = [0, 0, 0, 0]` (home)

`Pose_2 = [0, 55, -60, 0]` (extended)

`Pose_3 = [0, -80, 70, 0]` (retracted)

Error Type	Pose 1 Positional Error	Pose 2 Positional Error	Pose 3 Positional Error
Shoulder joint + 10°			
Elbow joint + 10°			
L2 length + 5cm			
L4 length + 5cm			

Question: Identify which joint and link errors cause the largest overall positional discrepancy. Why does an error at one specific joint or link scale differently than another? Based on this behavior, what is one additional distinct pose you would test to explore the absolute worst-case positional error?

Exercise 7:

Answer the following:

1. While cubic spline trajectories are convenient and straightforward to implement, what are their main mechanical and kinematic drawbacks? (Consider higher-order derivatives such as jerk at knot points and the physical strain on robot actuators).

2. In the `integration_err.py` demo, open-loop numerical integration caused the arm's trajectory to drift over time. Explain what causes this cumulative error in digital systems, and explain how the feedback term in `v_cmd = spline_velocity(t) + Kp * (spline(t) - location)` counteracts it.

8 Submission

Make sure to show the TA everything marked as such.

On Canvas:

- Submit your `forward_kin.py`.
- Submit `trajectory_generator.py` and a video showing it working
- Answer all questions and submit your worksheet
- Individual Reflection (see below)

8.0.1 Individual Reflection

While the codebase is a collaborative group effort, **each student must submit a standalone, individual reflection document** (maximum 1 page).

1. Specify your group's workflow and the role each member played.
2. Detail at least one specific technical issue or unpredictable behavior your group encountered. Explain how your group identified and resolved it.
3. Briefly describe everything covered in this lab (DH convention, forward kinematics, velocity kinematics) and why we care about them.

Note: To prevent passive participation, TAs will cross-reference these notes against group submissions. Identical or closely paraphrased responses within a group will result in a grade of zero for the individual component.